

Dar-Soins Maroc

Plateforme de Soins Infirmiers à Domicile

ANALYSE DÉTAILLÉE DU PROJET

Niveau Business | **Niveau Technique** | **Niveau Code**

Dépôt : github.com/goldenfox1407/dar-soins

Date du rapport : 2 juillet 2026

📑 Table des Matières

1. Niveau Business — Modèle et Domaine

1.1 Qu'est-ce que Dar-Soins ?

1.2 Modèle Économique

1.3 Rôles Utilisateurs et Flux Métier

1.4 Cycle de Vie d'une Intervention

1.5 Modules ERP

2. Niveau Technique — Architecture

2.1 Architecture Globale

2.2 Stack Technologique

2.3 Sécurité

2.4 Surface de l'API

2.5 Pipeline CI/CD

3. Niveau Code — Qualité et Structure

3.1 Structure Backend

3.2 Structure Frontend

3.3 Point Forts

3.4 Problèmes et Risques

3.5 Statistiques du Projet

4. Évaluation Synthétique

5. Statut Git et Recommandations

🏢 Niveau Business

1.1 — Qu'est-ce que Dar-Soins ?

Dar-Soins est une **plateforme SaaS de santé** connectant les patients au Maroc avec des infirmiers à domicile, supervisée par un back-office administratif. Il s'agit d'une solution globale combinant :

- Un **marketplace de soins à domicile** (mise en relation patient ↔ infirmier par géolocalisation)
- Un **module ERP intégré** (gestion des stocks, ventes, RH, finances)

1.2 — Modèle Économique

Composant	Description
Service	Soins infirmiers à domicile (injections, pansements, garde de nuit)
Clients	Patients nécessitant des soins à domicile
Prestataires	Infirmiers inscrits et validés sur la plateforme
Revenus	Facturation par intervention (frais de service + frais de déplacement + consommables + TVA 20%)
Administration	Gestionnaire (gère le personnel, les stocks, les finances)

1.3 — Rôles Utilisateurs et Flux Métier



Client

Commande un soin → reçoit un devis → approuve → visite de l'infirmier → paiement → facture PDF



Infirmier

Reçoit les missions → accepte/refuse → pointage sur site (photo) → enregistre les constantes → clôture l'intervention → signature patient

Gestionnaire (Admin)

ERP complet — produits, stocks, achats, ventes, RH, finances, rôles et permissions

1.4 — Cycle de Vie d'une Intervention

```
Client commande un soin
↓
Géolocalisation → Recherche de l'infirmier le plus proche
↓
L'infirmier accepte ou refuse la mission
↓
Génération automatique d'un devis (tarif soin + frais de déplacement + TVA)
↓
Le client approuve le devis
↓
L'infirmier arrive sur site (pointage par photo / selfie)
↓
Enregistrement des constantes médicales + consommables utilisés
↓
Signature du patient + choix du mode de paiement
↓
Génération automatique de la facture finale (PDF)
```

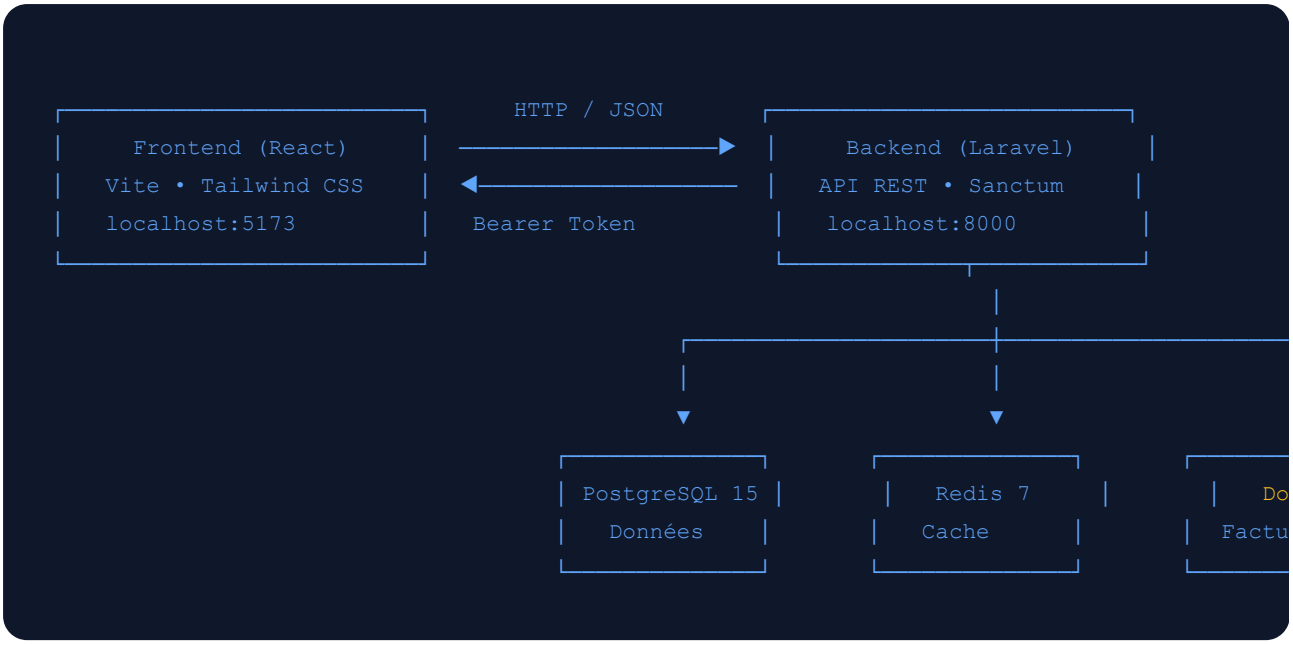
1.5 — Modules ERP (Domaines Fonctionnels)

Module	Fonctionnalités
Contacts	Clients, Fournisseurs
Inventaire	Produits, Catégories, Marques, Unités, Entrepôts, Variations
Achats	Bons de commande, Lignes d'achat, Retours fournisseurs
Ventes	Ventes, Lignes de vente, Retours clients, Devis
RH	Employés, Présences, Congés, Paie
Finance	Comptes bancaires, Dépenses, Transferts financiers, Devises

Module	Fonctionnalités
Système	Rôles, Permissions, Taxes, Rapports IA

🛠️ Niveau Technique

2.1 — Architecture Globale



2.2 — Stack Technologique

Couche	Technologie
Backend	PHP 8.2, Laravel 12, Sanctum (auth), Spatie Permission (RBAC), DomPDF
Frontend	React 19, Vite 8, React Router 7, Tailwind CSS 3, Axios, Leaflet (cartes), Framer Motion, react-signature-canvas
Base de données	PostgreSQL 15 (production), SQLite en mémoire (tests)
Cache	Redis 7 (production), database/array (local)
DevOps	Docker (Apache + Nginx), GitHub Actions CI/CD, Prometheus/Grafana
Tests	Pest PHP 3 (backend), Playwright (E2E), ESLint (frontend)

2.3 — Couche de Sécurité

Couche	Mécanisme	Statut
Authentification	Sanctum (tokens API)	✅ Implémenté
Autorisation	Spatie RBAC + vérification de rôle dans les contrôleurs	⚠️ Partiel — routes ERP sans middleware de rôle
Rate Limiting	60 req/min (API), 5 req/min (auth)	✅ Implémenté
CORS	Restreint à localhost:5173	✅ Configuré
Hachage	bcrypt (mots de passe)	✅ Implémenté

2.4 — Surface de l'API (40+ points d'accès)

Catégorie	Endpoints clés
Public	`/register`, `/login`, `/produits`
Interventions	commander, clôturer, affecter, devis, arrivée sur site, réponse client/infirmier
Factures	Liste, génération PDF
ERP CRUD	Users, Roles, Permissions, Clients, Fournisseurs, Catégories, Marques, Unités, Variations, Entrepôts, Achats, Ventes, Employés, Présences, Comptes bancaires, Transactions financières, Devis

2.5 — Pipeline CI/CD (GitHub Actions)

⚙️ Configuration CI/CD

Backend : PHP 8.2 + PostgreSQL + Redis → `composer install` → migrations → tests Pest avec `--coverage --min=80`

Frontend : Node 20 → `npm ci` → lint → build

Déclencheurs : push sur `main` / `develop`, PR vers `main`

📝 Niveau Code

3.1 — Structure Backend (26 Contrôleurs, 24 Modèles, 4 Services)

Contrôleurs orientés domaine métier

- **AuthController** — inscription / connexion / profil avec création de profil selon le rôle
- **InterventionController** — cycle complet : commande, affectation, devis, arrivée, clôture
- **ProduitController** — CRUD avec mise en cache Redis via `CacheService`
- **FactureController** — liste des factures + génération PDF (DomPDF)
- **EmployeController**, **TransactionFinanciereController**, etc. — CRUD générique (scaffoldé)

Services (couche logique métier)

- **ServiceTarification** — calcule les devis et factures (remise + déplacement + consommables + TVA)
- **ServiceAffectation** — trouve l'infirmier disponible le plus proche
- **ServiceGeolocalisation** — formule de Haversine pour le calcul de distance
- **CacheService** — wrapper Redis pour la mise en cache

> ⚠️ **Note importante**
>
> La plupart des modèles utilisent `$guarded = []` (mass-assignment ouvert) — un risque de sécurité pour les contrôleurs ERP scaffoldés.

3.2 — Structure Frontend (~50 pages)

- **Pages publiques** : Accueil, Services, À propos, FAQ, Contact, Connexion, Inscription
- **Espace Client** : Commander un soin, consulter les interventions, télécharger les factures
- **Espace Infirmier** : Missions (accepter/refuser), intervention active (constantes, consommables, signature, paiement), missions à venir (carte + pointage), historique
- **ERP Admin** (~40 pages) : Interface de gestion complète avec thème sombre glassmorphism, navigation latérale, tables avec recherche, modales

3.3 — Points Forts

✅ Atouts du projet

- Séparation claire des responsabilités (Contrôleurs → Services → Modèles)
- API Resources pour une sortie JSON cohérente et structurée
- Rate limiting et authentification dès le départ
- Mise en cache Redis sur les listes de produits
- Formule de Haversine pour le géocollaboration patient ↔ infirmier
- Interface responsive avec Tailwind CSS + Framer Motion
- Dockerisé pour le développement et la production
- Bonne couverture de tests sur les flux principaux (Auth, Produits, Interventions)

3.4 — Problèmes et Risques Identifiés

#	Problème	Sévérité	Détails
1	**Vulnérabilité mass-assignment**	CRITIQUE	La plupart des modèles utilisent `$guarded = []` ; les contrôleurs ERP utilisent `$request->all()` sans validation
2	**Middleware de rôle manquant sur les routes ERP**	CRITIQUE	Toutes les routes ERP CRUD sont derrière `auth:sanctum` uniquement — tout utilisateur connecté (y compris les clients) peut accéder aux fonctions admin
3	**Aucune validation d'entrée dans les contrôleurs ERP**	CRITIQUE	`EmployeController::store()` → `Employe::create($request->all())` — aucune règle de validation
4	**Scripts temporaires dans le dépôt**	MOYEN	`check_interventions.php` , `fill_migrations.php` , `fill_models.php` , `restore_intervention.php` , `test_cloturer.php` laissés à la racine du backend
5	**Fichier docker-compose.yml**	MOYEN	Le README référence `docker-compose.yml` / `docker-compose.prod.yml` mais ils n'existent pas

#	Problème	Sévérité	Détails
	manquant		dans le dépôt
6	**Tests frontend manquants**	MOYEN	Le README mentionne `npm test` (Jest) mais aucun script de test ni fichier de test n'existent dans le frontend
7	**URL API codée en dur**	FAIBLE	`api.js` utilise `baseURL: 'http://localhost:8000/api'` — devrait utiliser une variable d'environnement
8	**Test d'intervention obsolète**	FAIBLE	Le test attend `devis` et `distance_infirmier_km` dans la réponse de `commanderSoin` mais le contrôleur actuel ne les renvoie pas
9	**.env.example utilise SQLite**	FAIBLE	La CI utilise PostgreSQL mais `.env.example` est par défaut sur SQLite — incohérent
10	**VentesList récupère des interventions**	FAIBLE	`VentesList.jsx` récupère `/ventes` mais affiche des statuts d'intervention — inadéquation sémantique

3.5 — Statistiques du Projet

26 CONTRÔLEURS	**24** MODÈLES	**33** MIGRATIONS
~50 PAGES FRONTEND	**40+** ENDPOINTS API	**10** COMMITS

Composition détaillée

- **Backend** : 26 contrôleurs, 24 modèles, 6 API Resources, 4 services, 33 migrations, 3 fichiers de tests, 1 classe Mail
- **Frontend** : ~50 composants de pages, 4 composants partagés, 1 wrapper de service API

- **DevOps** : 2 Dockerfiles, config nginx, 1 workflow CI, config Prometheus/Grafana
- **Documentation** : 4 fichiers markdown de walkthrough + plan technique

🎯 Évaluation Synthétique

Dar-Soins est un **projet ambitieux et bien architecturé** qui combine un marketplace de soins de santé avec un ERP complet. Les fondations sont solides : authentification par rôles, géo-matching, logique de facturation, Docker, et CI/CD sont tous en place. Le frontend est soigné avec un design glassmorphism moderne.

Cependant, il existe des **lacunes de sécurité critiques** dans la couche ERP (mass-assignment, absence de vérification de rôle, aucune validation sur les contrôleurs scaffoldés) qui doivent être corrigées avant une utilisation en production. Les modules ERP semblent avoir été générés rapidement et nécessitent un durcissement.

📊 Score global

Architecture & Design	★★★★☆
Fonctionnalités métier (core)	★★★★☆
Modules ERP	★★★☆☆
Sécurité	★★☆☆☆
Tests & CI/CD	★★★☆☆
Documentation	★★★★☆

Statut Git et Recommandations

5.1 — Statut Git

Élément	Valeur
Dépôt	`https://github.com/goldenfox1407/dar-soins.git`
Branche active	`master` (10 commits)
Arbre de travail	Propre (aucune modification non commitée)
Branche principale	`master` (et non `main`)

5.2 — Recommandations Priorisées

🔴 Priorité 1 — Sécurité (avant production)

1. Ajouter `$fillable` sur tous les modèles en remplacement de `$guarded = []`
2. Ajouter un middleware de rôle (`role:gestionnaire`) sur toutes les routes ERP
3. Ajouter des règles de validation (`$request->validate([...])`) dans tous les contrôleurs ERP

🟡 Priorité 2 — Nettoyage & Cohérence

1. Supprimer les scripts temporaires (`check_interventions.php` , `fill_*.php` , etc.)
2. Créer les fichiers `docker-compose.yml` et `docker-compose.prod.yml` manquants
3. Corriger le test d'intervention (`InterventionTest.php`) pour qu'il corresponde à la réponse du contrôleur
4. Utiliser une variable d'environnement pour l'URL de l'API dans `api.js`

🟢 Priorité 3 — Améliorations

1. Ajouter des tests frontend (Vitest/Jest)

2. Aligner `.env.example` sur PostgreSQL pour la cohérence avec la CI
3. Corriger la sémantique de `VentesList` (ventes vs interventions)
4. Envisager l'internationalisation (i18n) pour le frontend

Dar-Soins Maroc

Analyse générée le 2 juillet 2026 • contact@darsoins.ma • https://darsoins.ma

Développé avec ❤️ au Maroc